TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 6
Seguridad

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · OWASP Mobile Top 10 (2024)

UNIDAD 6

Qué vamos a ver

Objetivos

- Entender por qué una app es más expuesta que una web.
- Guardar datos sensibles donde corresponde.
- Resolver un login sin manejar contraseñas.

Temas

- OWASP Mobile Top 10
- Almacenamiento seguro y tokens
- Biometría
- **OAuth 2.0 con PKCE**
- WebViews y navegador

UNIDAD 6 · PARTE 1

Por qué en móvil es distinto

Lo que cambia respecto de una aplicación web.

El binario está en manos del atacante

En web, el código que importa corre en tu servidor. En móvil, **la aplicación entera se descarga al dispositivo** — incluido el de alguien que quiera abrirla.

Lo que se puede hacer con tu APK

Descomprimirlo, leer el bundle de JavaScript, buscar cadenas, interceptar el tráfico con un proxy, correrlo en un emulador modificado.

La consecuencia práctica

Nada secreto puede vivir en la app. Ni una clave de API, ni una contraseña, ni un `client secret`. Aunque esté en una variable de entorno: termina en el bundle.

> Ofuscar no es cifrar. Retrasa a un curioso; no detiene a nadie con intención.

Cuatro diferencias con el escritorio

Redes que no controlás

El wifi del bar, el del aeropuerto, uno que se llama igual que el de la facultad. Toda la comunicación viaja por ahí.

Hardware sensible

Cámara, micrófono, GPS, biometría. Cada permiso que pedís es una puerta que abrís y que hay que justificar.

El dispositivo se pierde

Un teléfono robado o rooteado da acceso físico al almacenamiento de la app.

El parche pasa por la tienda

Publicar un arreglo lleva días de revisión, más los usuarios que nunca actualizan.

> De la última se desprende algo que vimos en la Unidad 7: **EAS Update** permite corregir el JavaScript sin pasar por la tienda. No todo se arregla así, pero para un bug de seguridad en la capa JS cambia los días por minutos.

UNIDAD 6 · PARTE 2

OWASP Mobile Top 10

El mapa de lo que efectivamente sale mal.

Qué es

OWASP

Fundación sin fines de lucro dedicada a la seguridad del software. Su material es abierto y es el estándar de facto de la industria.

El Top 10 móvil

Una lista de los diez riesgos más frecuentes y de mayor impacto en aplicaciones móviles, armada a partir de reportes de incidentes, CVEs y programas de bug bounty.

La lista vigente es la de 2024 y es bastante distinta de la de 2016, que todavía es la que aparece primero en muchas búsquedas. Si un artículo habla de «M2: Insecure Data Storage», está usando la vieja.

Los diez, edición 2024

M1 Uso indebido de credenciales
M2 Cadena de suministro insegura
M3 Autenticación / autorización insegura
M4 Validación de entrada y salida insuficiente
M5 Comunicación insegura
M6 Controles de privacidad insuficientes
M7 Protección del binario insuficiente
M8 Configuración de seguridad incorrecta
M9 Almacenamiento de datos inseguro
M10 Criptografía insuficiente

> Fijate que **M1, M2 y M3 encabezan la lista**: credenciales, dependencias y autenticación. No son fallas exóticas — son las tres cosas que todo TP integrador hace.

Los cuatro que les van a tocar

M1 · Credenciales

La clave de la API escrita en el código, o el token guardado sin cifrar. **Lo resolvemos en la Parte 3.**

M2 · Cadena de suministro

Una dependencia de npm comprometida o abandonada. `npm audit`, y no instalar paquetes con tres descargas.

M3 · Autenticación

Login casero, sesiones que no expiran, autorización decidida en el cliente. **Parte 4.**

M5 · Comunicación

HTTP en lugar de HTTPS, o certificados que no se validan. **Parte 5.**

UNIDAD 6 · PARTE 3

Dónde se guarda cada cosa

Almacenamiento seguro, tokens y biometría.

Lo que ya usamos no está cifrado

`expo-sqlite/kv-store`

Lo vimos en la Unidad 5 y está perfecto para preferencias, el último filtro elegido o el modo oscuro. **No está cifrado:** en un teléfono rooteado se lee en texto plano.

`expo-secure-store`

Para lo que no puede leerse nunca: tokens, credenciales, datos personales. Usa el almacén de claves del sistema operativo.

> La pregunta para decidir es simple: **si alguien lee este valor, ¿puede hacerse pasar por el usuario o acceder a sus datos?** Si la respuesta es sí, va en SecureStore.

expo-secure-store

```
import * as SecureStore from 'expo-secure-store';

await SecureStore.setItemAsync('token', valor);

const token = await SecureStore.getItemAsync('token');  // null si no hay

await SecureStore.deleteItemAsync('token');
```

Android

`SharedPreferences` cifradas con el **Keystore** del sistema.

iOS

Keychain, como `kSecClassGenericPassword`.

> Sólo guarda **cadenas de texto**. Un objeto va con `JSON.stringify`.

Tres detalles que sorprenden

Persistencia asimétrica

En **Android** los datos se borran al desinstalar. En **iOS** el Keychain los conserva y reaparecen si se reinstala la app con el mismo bundle ID.

Valores grandes

Expo no impone un límite, pero la plataforma puede rechazar valores muy grandes — históricamente iOS rechazaba por encima de ~2 KB. Hay que manejar el error.

No es una base de datos

Es un llavero. Para volumen, va SQLite; si además tiene que estar cifrado, una base con cifrado.

> Que en iOS el dato sobreviva a la desinstalación no es una garantía sobre la que apoyarse — es un detalle de implementación del Keychain. Tampoco es lo que espera un usuario que desinstala para «borrar todo».

Los dos tokens

Access token

- Vida corta: minutos u horas.
- Viaja en cada request, en la cabecera `Authorization`.
- Si se filtra, el daño tiene fecha de vencimiento.

Refresh token

- Vida larga: días o meses.
- Sólo sirve para pedir un access token nuevo.
- Si se filtra, el atacante se renueva solo. **Es el que más hay que cuidar.**

> Los dos van en SecureStore. Y el **cierre de sesión** tiene que borrarlos y además avisarle al servidor, para que revoque el refresh token: borrarlo sólo del teléfono no lo invalida.

Biometría

```
import * as LocalAuthentication from 'expo-local-authentication';

const hayLector = await LocalAuthentication.hasHardwareAsync();
const hayHuellas = await LocalAuthentication.isEnrolledAsync();

if (hayLector && hayHuellas) {
  const r = await LocalAuthentication.authenticateAsync({
    promptMessage: 'Confirmá tu identidad',
    cancelLabel: 'Cancelar',
  });
  if (r.success) abrirSesion();
}
```

> Siempre hay que preguntar **antes** si el dispositivo tiene lector y si el usuario tiene algo registrado. Y siempre tiene que haber un camino alternativo: no todos los teléfonos tienen biometría, y un dedo mojado no la pasa.

Biometría para abrir el llavero

```
await SecureStore.setItemAsync('refresh_token', valor, {
  requireAuthentication: true,
  authenticationPrompt: 'Desbloqueá para continuar',
});
```

Qué cambia

El dato no se puede leer sin que el usuario se autentique. La biometría deja de ser una pantalla decorativa y pasa a proteger el dato de verdad.

El costo

Si el usuario agrega una huella nueva o cambia el rostro registrado, la clave se invalida y **el valor deja de poder leerse**. Hay que preverlo.

> `requireAuthentication` **no funciona en Expo Go** cuando hay biometría disponible, porque falta el `NSFaceIDUsageDescription`. Va con development build y el plugin de `expo-secure-store`.

UNIDAD 6 · PARTE 4

Login con OAuth 2.0 y PKCE

Cómo entrar sin que la app toque nunca una contraseña.

Por qué no pedir la contraseña

Login propio

La app ve la contraseña. Tenés que guardarla, transmitirla, hashearla bien, resolver el «olvidé mi clave», el segundo factor, el bloqueo por intentos. Cada uno es una oportunidad de equivocarse.

OAuth 2.0

El usuario se autentica **en el sitio del proveedor** —Google, GitHub, el campus— dentro del navegador del sistema. Tu app nunca ve la contraseña: recibe un token.

> Y como el login ocurre en el navegador y no en una pantalla tuya, el usuario puede **verificar el dominio en la barra de direcciones**. Ese es justamente el control que una pantalla de login dentro de una WebView le quita.

El flujo, en cinco pasos

1
La app abre el navegador en el proveedor

2
El usuario se loguea y autoriza los permisos

3
El proveedor redirige a la app con un **código**

4
Ese código se canjea por un token

5
El token va a SecureStore

> Se llama authorization code grant. El código intermedio existe para que el token nunca viaje por la barra de direcciones del navegador, donde quedaría en el historial y en los logs.

Qué agrega PKCE

Queda un problema: otra app instalada en el mismo teléfono puede registrar el mismo esquema de redirección e interceptar el código del paso 3.

1 · Antes de empezar
La app inventa un secreto al azar (code verifier) y manda su **hash** al proveedor.

2 · Vuelve el código
El proveedor lo asocia a ese hash.

3 · Al canjear
Hay que presentar el secreto original. Quien robó el código no lo tiene y el canje falla.

En `expo-auth-session` viene activado por defecto (`usePKCE: true`). No hay que hacer nada — pero sí hay que saber que está, y no desactivarlo.

En código

```
import * as WebBrowser from 'expo-web-browser';
import { makeRedirectUri, useAuthRequest } from 'expo-auth-session';

WebBrowser.maybeCompleteAuthSession();   // cierra la ventana al volver

const discovery = {
  authorizationEndpoint: 'https://github.com/login/oauth/authorize',
  tokenEndpoint: 'https://github.com/login/oauth/access_token',
};

const [request, response, promptAsync] = useAuthRequest(
  { clientId: CLIENT_ID, scopes: ['identity'],
    redirectUri: makeRedirectUri({ scheme: 'tuapp' }) },
  discovery
);

<Button title="Entrar" disabled={!request} onPress={() => promptAsync()} />
```

Las cuatro reglas

`maybeCompleteAuthSession()`

Si falta, la ventana del navegador no se cierra al volver y el usuario queda mirando una pantalla en blanco.

`makeRedirectUri()`

No escribas la URL de redirección a mano: cambia entre desarrollo, build nativa y web.

`disabled={!request}`

El armado del pedido es asíncrono. Si el botón se aprieta antes, el navegador móvil bloquea la apertura.

Expo Go no sirve acá

No permite personalizar el esquema de la app, así que la redirección no vuelve. Hace falta un **development build**.

El `client secret` no va en la app

Varios proveedores piden un `client_secret` para canjear el código por el token. Ese canje **no puede ocurrir en el teléfono**: ya vimos que cualquier cadena del bundle es legible.

Dónde va

En un servidor tuyo. Puede ser una API route del mismo proyecto Expo, o el backend que ya tenga el grupo.

Qué hace la app

Manda el código a tu servidor, y tu servidor devuelve el token. El secreto nunca sale de ahí.

> Es otra vez el mismo principio de la Parte 1: **el cliente no es un lugar seguro**. Todo lo demás de esta unidad son casos particulares de eso.

UNIDAD 6 · PARTE 5

Contenido web y red

WebViews, navegador del sistema y HTTPS.

WebViews

Un componente que muestra una página web dentro de la app, con `react-native-webview`. Sirve para incrustar contenido que ya existe en web: términos y condiciones, un reporte, una pasarela de pago.

Los riesgos

- **XSS** si el contenido no es de confianza.
- **Phishing:** adentro no hay barra de direcciones, así que el usuario no puede verificar dónde está.
- **Redirecciones** a cualquier dominio.

Cómo se acota

- Cargar sólo **orígenes propios**.
- Bloquear la navegación a otros dominios.
- Desactivar lo que no se use.

> **Nunca un login dentro de una WebView.** Le quita al usuario la única forma que tiene de saber a quién le está dando la contraseña. Para eso está la Parte 4.

Acotar una WebView

```
<WebView
  source={{ uri: 'https://midominio.edu.ar/terminos' }}
  originWhitelist={['https://midominio.edu.ar']}
  javaScriptEnabled={false}
  onShouldStartLoadWithRequest={(req) =>
    req.url.startsWith('https://midominio.edu.ar')
  }
/>
```

> `onShouldStartLoadWithRequest` se ejecuta antes de cada navegación: si devolvés `false`, no carga. Es el control que evita que una redirección saque al usuario a otro dominio sin que se note.

Casi siempre alcanza con el navegador

```
import * as WebBrowser from 'expo-web-browser';

await WebBrowser.openBrowserAsync('https://uner.edu.ar');
```

Qué abre

Android: Chrome Custom Tabs. **iOS:** `SFSafariViewController`. En los dos casos, con barra de direcciones visible.

Por qué es más seguro

El contenido corre fuera de tu app, con el motor y los parches del sistema. Y el usuario ve el dominio.

> Regla práctica: **si el contenido no es tuyo, va al navegador**. La WebView se justifica cuando necesitás que se vea integrado y controlás el origen.

HTTPS, y qué es realmente el pinning

Lo obligatorio

HTTPS siempre. Android e iOS ya bloquean el tráfico en texto plano por defecto. Si algo «no anda» y la solución que encontrás en internet es habilitar HTTP, la solución es otra.

Certificate pinning

La app acepta **un certificado concreto**, no cualquiera que la cadena de confianza valide. Protege contra un certificado emitido de forma fraudulenta o un proxy instalado en el dispositivo.

Tiene un costo real: **el día que rote el certificado del servidor, todas las apps instaladas dejan de conectarse** hasta que publiques una versión nueva. Se justifica en banca y salud; en un TP integrador, no.

Permisos: pedir sólo lo que se usa

```
// cada módulo pide lo suyo, cuando hace falta
const { status } = await Location.requestForegroundPermissionsAsync();
if (status !== 'granted') return;
```

No existe más `expo-permissions`. Ese paquete se retiró hace varias versiones: hoy cada módulo expone su propio `requestPermissionsAsync()`, y lo declarativo va en el `app.json`.

Pedir permisos de más no es sólo un riesgo (M6): **las tiendas rechazan apps** que piden permisos sin justificación, y iOS exige explicar cada uno en el `Info.plist`.

UNIDAD 6

Para llevarse

- **El cliente no es un lugar seguro.** Todo lo demás de esta unidad se deduce de ahí.
- Tokens en **SecureStore**, nunca en kv-store.
- Login por **OAuth con PKCE**, en el navegador del sistema y no en una WebView.
- Los secretos y la validación que importa, **en el servidor**.

`owasp.org/www-project-mobile-top-10` · `docs.expo.dev/guides/authentication`